

THE LAST PROTOCOL

ENGINEERING NOTEBOOK — RECOVERED

The following pages were recovered from the calibration engineer's private notebook.

1. BLOCK STRUCTURE

The archive transformation operates on 64-bit blocks. Each block is divided into two 32-bit halves.

The transformation is a Feistel network. Its structure allows the transformation to be reversed when the round keys are known.

Notation:

L_r = left half entering round r
 R_r = right half entering round r
 K_r = round key for round r

The round relationship is:

$L_{(r+1)} = R_r$
 $R_{(r+1)} = L_r \text{ XOR } F(R_r, K_r, r)$

The archive uses eight rounds. The round keys are not stored alongside the ciphertext.

2. ROUND MATERIAL

The transformation uses eight distinct 256-bit round keys. The round function consumes the first eight bytes of each key.

A. The recovered history

The ordered state history recovered from the transmission and fragments is the foundation of the cryptographic material. Preserve the correct order; a different ordering produces different derived values.

B. Representation

```
State values: decimal integers
State separator: colon (:)
Text encoding: UTF-8
Digest algorithm: SHA-256
```

C. Root and chained material

The protocol identifier and the serialized state history contribute to the initial cryptographic root. The state history also contributes a separate digest.

Eight round keys are derived from this material. The schedule is chained: each round depends on the preceding stage and round-specific information.

The state history is input material, not a list of round keys. Determine the exact derivation by combining the clues in this notebook with the other supplied artifacts.

D. Round-key interpretation

```
k = int.from_bytes(K_r[:8], 'big')
key_low = k & 0xffffffff
key_high = (k >> 32) & 0xffffffff
```

Each key is 32 bytes. Its first eight bytes are interpreted as a big-endian integer and split into two 32-bit words for the round function.

The root is derived using SHA-256 over a protocol-specific prefix and the serialized state history. A separate SHA-256 digest of the state history is also used. Each round key is derived by hashing the preceding key material together with the round number and the state-history digest. The round number is encoded as a two-byte big-endian integer.

3. ROUND FUNCTION

The function F accepts a 32-bit right word, a 32-byte round key, and a round number r . Intermediate arithmetic uses unsigned 32-bit words wherever a mask is specified.

Constants

Purpose	Value
32-bit mask	0xffffffff
Round constant	0x9E3779B9
Mixing constant	0x045D9F3B

Exact operation sequence

$\text{MASK}_{32} = 0xffffffff$

1. $k = \text{int.from_bytes}(K_r[:8], \text{'big'})$
2. $x = (R_r \text{ XOR } (k \text{ AND } \text{MASK}_{32}) \text{ XOR } ((k \gg 32) \text{ AND } \text{MASK}_{32})) \text{ AND } \text{MASK}_{32}$
3. $x = (x + r * 0x9E3779B9) \text{ AND } \text{MASK}_{32}$
4. $x = \text{ROTL}_{32}(x, (3 * r) \bmod 32)$
5. $x = x \text{ XOR } ((x \gg 13) \text{ OR } ((x \ll 19) \text{ AND } \text{MASK}_{32}))$
6. $x = (x * 0x045D9F3B) \text{ AND } \text{MASK}_{32}$
7. $x = x \text{ XOR } (x \gg 16)$
8. $F(R_r, K_r, r) = x \text{ AND } \text{MASK}_{32}$

Rotation definition

$\text{ROTL}_{32}(x, n) = ((x \ll n) \text{ OR } (x \gg (32 - n))) \text{ AND } \text{MASK}_{32}$

Rotate left by $(3 * r) \bmod 32$ bits. Reduce the rotation amount modulo 32 before applying the rotation.

Shift and masking details

In step 5, evaluate both shifted expressions using the value of x immediately before step 5. Combine the shifted expressions with bitwise OR, then XOR that result with x . Apply the masks exactly where specified.

Rounds are numbered from 1 through 8, inclusive.

4. BYTE ORDER AND BLOCK PROCESSING

All multi-byte values use network byte order (big-endian).

For an eight-byte input block:

```
left  = int.from_bytes(block[0:4], 'big')
right = int.from_bytes(block[4:8], 'big')
```

Round processing

```
for r = 1 through 8:
    new_left  = right
    new_right = (left XOR F(right, K_r, r)) AND MASK32
    left  = new_left
    right = new_right
```

After all eight rounds:

```
output = left.to_bytes(4, 'big')
        + right.to_bytes(4, 'big')
```

Do not swap the halves after the final round. The output is the final left half followed by the final right half, both in big-endian byte order.

The archive contains multiple blocks. Determine the original block boundaries and the padding convention from the supplied evidence before processing the archive.

5. KNOWN TEST VECTOR

The following block was recorded before the final archive was produced. Use it to verify the implementation of the block transformation.

Field	Value
Plaintext block	52 45 42 55 49 4C 44 3A
Ciphertext block	FA D9 24 EE CF 4A DE AF

Verification

The test block is exactly eight bytes. Do not add padding to this block before testing it.

If the result differs, check the recovered state order, key-schedule reconstruction, byte order, rotation direction, shift operations, and 32-bit masking.

"Verify the small block before processing the archive."

6. FINAL NOTE

The archive is not the final message.

The engineer wrote:

"The first successful decryption only tells you that the machine has been rebuilt."

"What it says next is another problem."

END OF NOTEBOOK